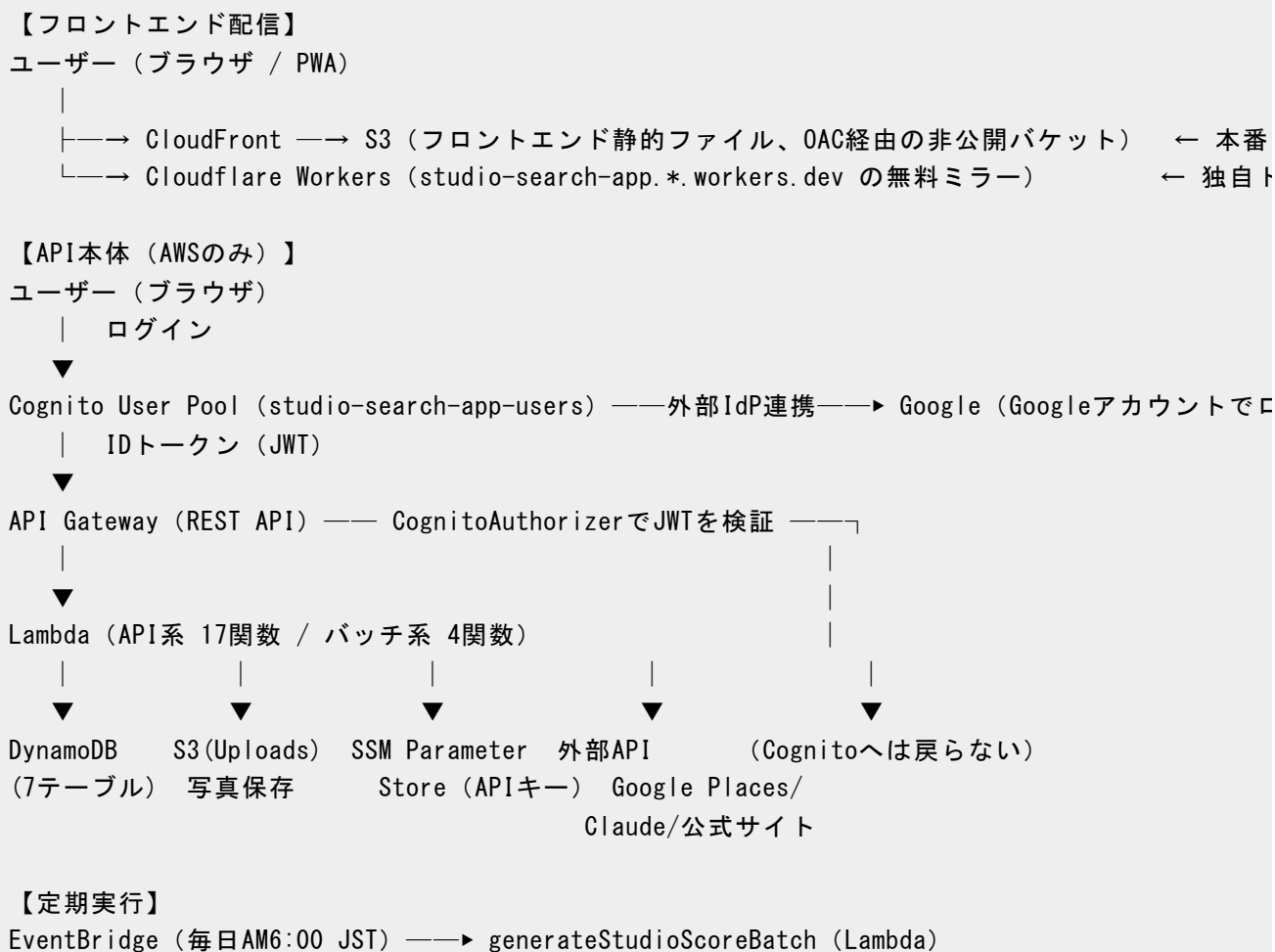


AWS構成・サービス連携ガイド（レンタルスタジオサーチアプリ）

このアプリで実際に使っているAWS（およびAWS外の関連）サービスを一覧化し、「何のために」「なぜそれを選んだか」「他のどのサービスと連携しているか」をサービス単位でまとめたもの。Lambda関数単位の詳細は[詳細設計.html](#)、ファイル単位の対応は[ファイル構成・依存関係.html](#)を参照。本資料は釣行AIアプリ（fishing-ai-app）と同じAWSアカウント上に、完全に独立したCloudFormationスタック（studio-search-app）として構築した時点の構成を反映している。

1. 全体構成図



【コスト監視】

AWS Budgets → メール通知（月\$10の80%/100%到達時）

2. サービス一覧（早見表）

サービス	役割	主な連携先
Amazon Cognito	Googleアカウントでのログイン認証（釣行AIアプリとは別のUser Pool）	Google（外部IdP）、API Gateway（JWT検証）
Amazon API Gateway	フロントからのHTTPリクエストの受付・認証・Lambdaへのルーティング	Cognito、Lambda（21関数）
AWS Lambda	実際の処理を行うプログラム本体（API系・バッチ系の2パッケージ）	DynamoDB、S3、SSM、外部API、他のLambda（非同期起動）
Amazon DynamoDB	アプリの全データ（スタジオ・おすすめ・投稿・チャット等）の保存先	Lambda
Amazon S3	①フロントエンドの静的ファイル配信 ②ユーザーがアップロードする写真の保存	CloudFront（OAC経由、①）、Lambda・ブラウザ直PUT（②）
Amazon CloudFront	フロントエンドをCDN経由で高速配信（本番の正系）	S3（Origin Access Control経由のオリジン）
Cloudflare Workers（AWS外）	フロントエンドの無料ミラー配信（独自ドメイン未取得のため）	フロントのビルド成果物を直接デプロイ（AWSとは独立、Worker名studio-search-app）
AWS Systems Manager Parameter Store（SSM）	外部APIキー（Claude・Google Places・Google OAuth）の安全な保管	Lambda（実行時に取得）。パスは/studio-search/*で釣行AIアプリ（/fishing-ai/*）とは分離
Amazon EventBridge	毎日決まった時刻にバッチLambdaを自動起動するスケジューラ	generateStudioScoreBatch（Lambda）
AWS Budgets	AWS利用料が閾値を超えたらメール通知	Eメール（rfunao0955@gmail.com）

AWS IAM	各Lambdaに必要最小限の権限だけを付与	全AWSサービスへのアクセス制御
GitHub Actions (AWS外)	mainブランチへのpushをトリガーに自動デプロイ	AWS (SAM CLI経由)、Cloudflare、S3、CloudFront

3. 各サービスの詳細

■ Amazon Cognito (User Pool + Google連携)

ログイン機能そのもの。「Googleアカウントでログイン」ボタンを押すとCognitoのHosted UIに飛び、Cognitoが裏でGoogleと認可コードのやり取りをして、最終的にアプリ向けのIDトークン（JWT）を発行する。このJWTがAPI Gatewayでの認証に使われる。Hosted UIドメインは `studio-search-app-<AWSアカウントID>.auth.ap-northeast-1.amazonaws.com`。

なぜ釣行AIアプリと別のUser Poolか: Client ID・Hosted UIドメインはUser Pool単位でしか発行できず、共有するとログイン画面のURLやリダイレクト設定がアプリ間で衝突する。Google Cloud Console側のOAuthクライアントも専用のものを別途発行している。

■ Amazon API Gateway (REST API)

フロントエンドが呼び出す全エンドポイント（`/recommendations`、`/chat`、`/posts`等）の窓口。ログイン必須のエンドポイントにはCognitoAuthorizerをアタッチしており、JWTが無効・未提示なら該当Lambdaを呼ばずに401を返す。

なぜAPI Gateway: Lambdaを直接インターネットに公開せず、認証・CORS・ルーティングを一括管理できる。統合タイムアウトが29秒固定という制約があり、これがAIチャット（同期呼び出し、Lambda側25秒に短縮）とAI分析（非同期呼び出しに変更）の設計判断に直接影響している（詳細設計.html参照）。

■ AWS Lambda (API系・バッチ系の2パッケージ)

実処理の本体。`backend/lambda/api/`（17関数、`handlers.py`）と `backend/lambda/batch/`（4関数、`generate_studio_score.py`等）の2つのSAM CodeUriパッケージに分かれており、この2つの間でコードは共有できない（`batch_common.py`のようなヘルパーが両パッケージに重複して存在する理由）。

なぜLambda（サーバーレス）：個人利用規模のトラフィックでは「常時起動のサーバー」は過剰投資になる。呼ばれた時だけ課金される従量課金と、インフラ管理不要な点を重視した。

Lambda関数名を固定しなかった理由：当初FunctionNameを 釣行AIアプリと同じ命名（getRecommendationsHandler等）で固定していたところ、Lambda関数名はリージョン内でアカウント共通の一意制約があるため、既に稼働している 釣行AIアプリの同名関数と衝突し、初回デプロイがAWS::EarlyValidation::ResourceExistenceCheck エラーで失敗した。全LambdaのFunctionName 指定を削除し、CloudFormationに 自動採番させる形に修正することで解消した。

■ Amazon DynamoDB（7テーブル）

テーブル名	役割
studio-studios	スタジオマスタ（座標・名前・設備・収容人数・最寄り駅距離・料金・写真URL等）
studio-recommendations	スタジオごとの最新スコア・AIコメント（1スタジオ1件、上書き型）
studio-favorites	お気に入り登録
studio-posts	レビュー投稿（本文・写真・★評価）
studio-chats	AI相談チャットの会話履歴
studio-usage	1日あたりのAI/API呼び出し回数（レート制限用、TTLで自動削除）
studio-users	ユーザーの表示名（ユーザー名）

なぜDynamoDB：スキーマがシンプルでアクセスパターンも単純（PK検索・全件scan中心）なため、RDBのような複雑なクエリは不要。PAY_PER_REQUESTモードで使った分だけ課金され、サーバーレスLambdaとの相性も良い。

■ Amazon S3（2バケット）

①フロントエンド配信用バケット（studio-search-app-frontend-＜アカウントID＞）：Reactのビルド成果物（HTML/JS/CSS）を置き、CloudFrontがOrigin Access Control（OAC）経由で参照する非公開バケット。②UploadsBucket（studio-search-app-uploads-＜アカウントID＞）：ユーザーが投稿・スタジオに設定する写真の保存先。フロントはLambda（presignUploadHandler）から発行される署名付きPOSTフォームを使ってブラウザから直接S3へアップロードし、Lambda自体は画像バイナリを経由しない（API Gatewayのペイロードサイズ制限回避のため）。

なぜOACで非公開バケットか: S3の静的website機能（公開バケット）ではなく、CloudFrontのOrigin Access Control経由でのみ読み取りを許可するバケットポリシーにしている。バケット自体への直接アクセスを防ぎつつ、CloudFront経由のアクセスだけを許可できる、より安全な構成のため（釣行AIアプリの既存構成を踏襲）。

なぜ署名付きアップロード: 画像バイナリをLambda経由にするとAPI Gatewayのペイロード上限に引っかかる。S3への直接アップロードにすることでこの制約を回避しつつ、content-length-range条件でアップロード上限（8MB）をS3側に強制させている。

■ Amazon CloudFront + Cloudflare Workers（フロント配信が2系統ある理由）

本番の正系はCloudFront（S3をオリジンとするCDN、OAC経由）。index.htmlはno-cache、ハッシュ付きアセット（/assets/index-`<hash>`.js）は長期キャッシュという構成。加えて、独自ドメインを取得する前の段階で見た目の良いURLを用意するため、Cloudflare Workers（studio-search-app.*.workers.dev）にも同じビルド成果物を並行デプロイしている。バックエンドAPI（API Gateway）はAWS側1本のみで共有しており、フロントの配信経路だけが2系統ある形。

釣行AIアプリのWorkerと衝突しない理由: wrangler.jsoncのnameをstudio-search-app（釣行AIアプリはryu-chan-fish）と明示的に別名にしており、GitHub Secrets（CLOUDFLARE_API_TOKEN等）もリポジトリごとに別登録のため、どちらかのアプリのデプロイがもう片方のWorkerを誤って上書きすることは無い。

■ AWS Systems Manager Parameter Store（SSM）

Claude APIキー（/studio-search/anthropic-api-key）、Google Places APIキー（/studio-search/google-places-api-key）、Google OAuthクライアントID/シークレット（/studio-search/google-oauth-client-id-secret）をSSMに保管。Lambdaの環境変数に直書きせず、実行時にssm:GetParameterで取得する。

なぜ釣行AIアプリと別パスか: AnthropicのAPIキー自体（クレジット残高）は釣行AIアプリと共有してよい前提だが、Lambdaごとの権限スコープ（IAMポリシーが参照するパラメータ名）を綺麗に保つため、登録パス自体は/fishing-ai/*とは分離している。実際、両アプリのAnthropicキー・Google Placesキーは同じ値を使い回して登録した。

■ Amazon EventBridge

`cron(0 21 * * ? *)` (UTC 21:00 = 日本時間AM6:00) で `generateStudioScoreBatch`を毎日自動起動し、全スタジオのスコアを最新化する。ユーザーが「AI分析を実行」ボタンを押さなくても、朝には最新の おすすめが用意されている状態を作るための仕組み。

■ AWS Budgets

月\$10のAWS利用料予算を設定し、80%/100%到達時にメールで通知する (`template.yaml`の`CostBudget`リソース、予算名 `studio-search-app-monthly`)。ただしこれはAWS側の課金のみを見ており、Google Cloud (Places/Maps) やAnthropic (Claude) の課金はAWSの外で発生するため対象外。それぞれGoogle Cloud ConsoleとAnthropic Console側で別途予算アラート・`spend limit`の 設定が必要 (README.md「10. 利用料アラート」参照)。

■ AWS IAM (最小権限方針)

各Lambda関数には、その関数が実際に必要とするDynamoDBテーブル・S3バケット・SSMパラメータへのアクセス権限だけをSAMのPolicies (`DynamoDBCrudPolicy`・`S3WritePolicy`・`SSMParameterReadPolicy`等) で個別に付与している。例えば `GetStudiosFunction`は`Studios`テーブルの読み取りしかできず、書き込みや他テーブルへのアクセス権限は持たない。

4. 主要フローのサービス連携トレース

① ログイン

ブラウザ「ログイン」クリック

- Cognito Hosted UI (`studio-search-app-<アカウントID>.auth.ap-northeast-1.amazoncognito.com`)
- Google認証画面 (Cognitoが仲介)
- Cognitoに戻りIDトークン (JWT) 発行
- フロントがJWTを保持し、以後のAPIリクエストにAuthorizationヘッダーとして付与

② 「AI分析を実行」ボタン

- フロント → API Gateway → `adminRunAiBatch` (Lambda、UsageTableでレート制限チェック)
- `generateStudioScoreBatch` を非同期起動して即202を返す
 - `generateStudioScoreBatch` が裏側で実行:
StudiosTable全件取得

→ `discover_studios.run_discovery()` で新規スタジオ探索 (Google Places API、SSMからキー取得)
新規候補ごとに: 設備推測・収容人数推測 (Claude)、最寄り駅検索 (Places Nearby Search
公式サイト取得+料金抽出 (Place Details→サイト取得→Claude))
→ スコア計算 (ルールベース、最寄り駅距離を使用)
→ スコアが大きく動いたスタジオのみClaude APIで理由文生成 (SSMからキー取得)
→ RecommendationsTableへ保存
→ フロントは GET /recommendations を数秒おきにポーリングして完了を検知

③ AI相談チャット (画像添付あり)

フロント → API Gateway (CognitoAuthorizerでJWT検証) → `postChatHandler`
→ UsageTableでレート制限チェック (1日30件)
→ S3(Uploads)から画像バイトを取得
→ StudiosTable・RecommendationsTableをscanして実データのコンテキストを構築
→ Claude APIへテキスト+画像+実データコンテキストを送信 (SSMからキー取得)
→ 応答をChatsTableに保存し、同期的にフロントへ返却

④ 写真アップロード (レビュー投稿・スタジオ写真)

フロント → `presignUploadHandler` (Lambda) が署名付きPOSTフォームを発行
→ ブラウザがS3(Uploads)へ直接POST (Lambda/API Gatewayを経由しない)
→ アップロード完了後、公開URLをPosts/StudiosテーブルにLambda経由で保存

5. なぜこの構成にしたか (設計方針まとめ)

- **サーバーレス徹底:** 個人利用規模のトラフィックに対して常時稼働のサーバーは過剰。使った分だけ課金されるLambda・DynamoDB・S3 (+CloudFront) で構成し、待機コストをゼロに近づけている
- **バッチ中心・リアルタイム処理の排除:** スコア計算はEventBridgeの日次バッチと手動実行の2経路に絞り、常時ポーリングするような設計を避けている
- **実データとAI生成の分離:** 座標・実在性・料金など「間違えられないデータ」はGoogle Places APIや公式サイトの実テキストのみを情報源とし、AI (Claude) は説明文生成や 設備・収容人数の推測など「間違っても実害が小さい付随情報」にのみ使う (ハルシネーション対策。料金抽出も「サイトに書いてあれば数字を返す、無ければ不明」であり AIが金額を作り出すことはない)
- **最小権限のIAM:** Lambdaごとに必要なリソースへのアクセス権限だけを絞って付与
- **コスト保護の多層化:** ①ユーザー単位の1日あたりレート制限 (UsageTable)、②AWS Budgets (AWS利用料)、③Google Cloud・Anthropic Console側の予算アラート (AWS外の課金)、という3層でコスト暴走を防ぐ設計にしている

- **既存アプリとの独立性:** 釣行AIアプリと同じAWSアカウント・同じ Anthropic/Cloudflare アカウントを使い回しつつも、CloudFormationスタック・Cognito User Pool・SSMパラメータパス・Cloudflare Worker名・GitHub Secretsは全て別物にし、互いの運用に影響が出ないようにしている